AdvR 03 – Subsetting

Selecting Multiple Elements

Six Ways to Subset Atomic Vectors (1–3)

```
x <- c(2.1, 4.2, 3.3, 5.4)
x[c(3, 1)]         # 1. Positive integers: select
```

```
#> [1] 3.3 2.1
```

```
x[c(1, 1)]         # Duplicates allowed
```

```
#> [1] 2.1 2.1
```

```
x[-c(3, 1)]        # 2. Negative integers: exclude
```

```
#> [1] 4.2 5.4
```

```
x[c(TRUE, TRUE, FALSE, FALSE)]   # 3. Logical vector
```

```
#> [1] 2.1 4.2
```

```
x[x > 3]           # Most common use
```

```
#> [1] 4.2 3.3 5.4
```

Six Ways to Subset Atomic Vectors (4–6)

```
x[]                 # 4. Nothing: returns all
```

```
#> [1] 2.1 4.2 3.3 5.4
```

```
x[0]                # 5. Zero: returns empty
```

```
#> numeric(0)
```

```
y <- setNames(x, letters[1:4])
y[c("d", "c", "a")]   # 6. Character: by name
```

```
#>   d   c   a
#> 5.4 3.3 2.1
```

Matrices and Data Frames

Matrices — subset with `[rows, cols]`:

```
a <- matrix(1:9, nrow = 3)
colnames(a) <- c("A", "B", "C")
a[1:2, ]
```

```
#>      A B C
#> [1,] 1 4 7
#> [2,] 2 5 8
```

```
a[c(TRUE, FALSE, TRUE), c("B", "A")]
```

```
#>      B A
#> [1,] 4 1
#> [2,] 6 3
#> ... [output truncated]
```

Data frames — single index = columns (list behavior); two indices = rows, columns (matrix behavior):

```
df <- data.frame(x = 1:3, y = 3:1, z = letters[1:3])
df[c("x", "z")]     # list-style: data frame
```

```
#>   x z
#> 1 1 a
#> 2 2 b
#> 3 3 c
```

Preserving Dimensionality

By default, [simplifies output (drops dimensions):

```
a <- matrix(1:4, nrow = 2)
str(a[1, ])               # vector (simplified)
```

```
#>  int [1:2] 1 3
```

```
str(a[1, , drop = FALSE]) # matrix (preserved)
```

```
#>  int [1, 1:2] 1 3
```

Same for data frames:

```
df <- data.frame(a = 1:2, b = 1:2)
str(df[, "a"])             # vector
```

```
#>  int [1:2] 1 2
```

```
str(df[, "a", drop = FALSE]) # data frame
```

```
#> 'data.frame':    2 obs. of  1 variable:
#>  $ a: int  1 2
```

Use `drop = FALSE` in functions to avoid surprising simplification.

Tibbles default to `drop = FALSE` — they always return tibbles.

Exercises (Selecting Multiple)

1. Fix each error:

```
mtcars[mtcars$cyl = 4, ]          # a
mtcars[-1:4, ]                     # b
mtcars[mtcars$cyl <= 5]            # c
mtcars[mtcars$cyl == 4 | 6, ]      # d
```

2. Why does `x[NA]` (where `x <- 1:5`) return five `NA`s?
3. What does `upper.tri()` return? How does subsetting a matrix with it work?
4. Why does `mtcars[1:20]` error but `mtcars[1:20, ]` doesn't?
5. Implement your own function to extract diagonal entries from a matrix.

Solutions (Selecting Multiple) — 1

```r
# a. Use == not =
mtcars[mtcars$cyl == 4, ]

# b. Use -(1:4) not -1:4
mtcars[-(1:4), ]

# c. Missing comma (selects columns, not rows)
mtcars[mtcars$cyl <= 5, ]

# d. Need full comparison on both sides
mtcars[mtcars$cyl == 4 | mtcars$cyl == 6, ]
# Or: mtcars[mtcars$cyl %in% c(4, 6), ]
```

2. `NA` is logical type → recycled to `c(NA, NA, NA, NA, NA)`. Each `NA` index produces `NA` output.
3. `upper.tri()` returns a logical matrix (`TRUE` above diagonal). Subsetting with a logical matrix selects all `TRUE` positions as a vector.
4. Single index on data frame = column selection. `mtcars[1:20]` tries to select 20 columns, but only 11 exist. Two-index `mtcars[1:20, ]` selects rows.
5.

```
diag2 <- function(x) {
  n <- min(nrow(x), ncol(x))
  x[cbind(seq_len(n), seq_len(n))]
}
diag2(matrix(1:9, 3))
```

```
#> [1] 1 5 9
```

Selecting a Single Element

[[— Extract One Element

[returns a **sub-structure** (list → list). [[extracts the **element itself**:

Train car metaphor:

- `x[4:6]` = a train of cars 4–6
- `x[[5]]` = the **contents** of car 5

```
x <- list(1:3, "a", 4:6)
str(x[1])      # list containing the vector
```

```
#> List of 1
#>  $ : int [1:3] 1 2 3
```

```
str(x[[1]])    # the vector itself
```

```
#>  int [1:3] 1 2 3
```

[[can only take a **single** positive integer or string.

Recursive indexing: `x[[c(1, 2)]]` = `x[[1]][[2]]` (dig into nested structures).

$ — Shorthand for Named Elements

`x$y` is roughly `x[["y"]]`, but with **partial matching**:

```
x <- list(abc = 1, def = 2)
x$a          # matches "abc" --- dangerous!
```

```
#> [1] 1
```

```
x[["a"]]     # NULL (exact match only)
```

```
#> NULL
```

Common mistake with data frames:

```
var <- "cyl"
# mtcars$var   # wrong! looks for column named "var"
mtcars[[var]]   # correct! uses the value of var
```

```
#>  [1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8
#> [26] 4 4 4 8 6 8 4
```

Disable partial matching: `options(warnPartialMatchDollar = TRUE)`.

Missing and Out-of-Bounds with [[

`row[[col]]`	Zero-length	OOB (int)	OOB (chr)	Missing
Atomic	Error	Error	Error	Error
List	Error	Error	`NULL`	`NULL`
`NULL`	`NULL`	`NULL`	`NULL`	`NULL`

Use `purrr::pluck()` for safe extraction with defaults:

```
x <- list(a = list(1, 2, 3), b = list(3, 4, 5))
purrr::pluck(x, "a", 1)    # 1
```

```
#> [1] 1
```

```
purrr::pluck(x, "c", 1)    # NULL (no error)
```

```
#> NULL
```

```
purrr::pluck(x, "c", .default = NA)  # NA
```

```
#> [1] NA
```

Exercises (Single Element)

1. Brainstorm as many ways as possible to extract the third value from `cyl` in `mtcars`.
2. Given `mod <- lm(mpg ~ wt, data = mtcars)`, extract the residual degrees of freedom. Then extract R-squared from `summary(mod)`.

Solutions (Single Element) — 1

1. Many approaches:

```
# Column first, then element
mtcars$cyl[[3]]
```

```
#> [1] 4
```

```
mtcars[["cyl"]][[3]]
```

```
#> [1] 4
```

```
# Row first, then column
mtcars[3, "cyl"]
```

```
#> [1] 4
```

```
mtcars[3, ]$cyl
```

```
#> [1] 4
```

Also: `mtcars[[c(2, 3)]]`, `with(mtcars, cyl[[3]])`.

2. Models are lists:

```
mod <- lm(mpg ~ wt, data = mtcars)
mod$df.residual
```

```
#> [1] 30
```

```
summary(mod)$r.squared
```

```
#> [1] 0.7528328
```

Subsetting and Assignment

Subassignment

Combine subsetting operators with assignment:

```
x <- 1:5
x[c(1, 2)] <- c(101, 102)
x
```

```
#> [1] 101 102   3   4   5
```

With lists — `NULL` removes, `list(NULL)` adds literal NULL:

```
x <- list(a = 1, b = 2)
x[["b"]] <- NULL     # removes b
str(x)
```

```
#> List of 1
#>  $ a: num 1
```

```
y <- list(a = 1, b = 2)
y["b"] <- list(NULL)  # b exists but is NULL
str(y)
```

```
#> List of 2
#>  $ a: num 1
#>  $ b: NULL
```

Subsetting with Nothing Preserves Structure

`x[]` keeps the original structure:

```
mtcars[] <- lapply(mtcars, as.integer)
is.data.frame(mtcars)  # TRUE --- still a data frame
```

```
#> [1] TRUE
```

Without `[]`:

```
data(mtcars)  # reset
mtcars2 <- lapply(mtcars, as.integer)
is.data.frame(mtcars2)  # FALSE --- now a list!
```

```
#> [1] FALSE
```

Replace NAs: `df[is.na(df)] <- 0` replaces all `NA`s with `0`.

Applications

Lookup Tables

A **named vector** acts as a simple lookup table:

```
x <- c("m", "f", "u", "f", "f", "m", "m")
lookup <- c(m = "Male", f = "Female", u = NA)
unname(lookup[x])
```

```
#> [1] "Male"   "Female" NA       "Female" "Female"
#> [6] "Male"   "Male"
```

For complex lookups, use `match()`:

```
grades <- c(1, 2, 2, 3, 1)
info <- data.frame(grade = 3:1,
  desc = c("Excellent", "Good", "Poor"),
  fail = c(FALSE, FALSE, TRUE))
info[match(grades, info$grade), ]
```

```
#>     grade      desc  fail
#> 3       1      Poor  TRUE
#> 2       2      Good FALSE
#> 2.1     2      Good FALSE
#> ... [output truncated]
```

For multiple columns, use `merge()` or `dplyr::left_join()`.

Random Samples and Ordering

Random sampling with `sample()`:

```
df <- data.frame(x = c(1, 2, 3, 1, 2), y = 5:1)
df[sample(nrow(df), 3), ]   # 3 random rows
```

```
#>   x y
#> 5 2 1
#> 4 1 2
#> 3 3 3
```

Ordering with `order()`:

```
x <- c("b", "c", "a")
x[order(x)]
```

```
#> [1] "a" "b" "c"
```

```
# Sort data frame by column
df[order(df$x), ]

# Sort columns alphabetically
df[order(names(df))]
```

Expanding and Deduplicating

Expanding aggregated counts with `rep()`:

```
df <- data.frame(x = c(2, 4, 1), n = c(3, 5, 1))
df[rep(1:nrow(df), df$n), ]
```

```
#>     x n
#> 1   2 3
#> 1.1 2 3
#> 1.2 2 3
#> 2   4 5
#> ... [output truncated]
```

Deduplicating with `duplicated()` or `!duplicated()`:

```
df[!duplicated(df$x), ]
```

Removing Columns and Filtering Rows

Remove columns — three ways:

```
df$z <- NULL                        # set to NULL
df[c("x", "y")]                     # keep wanted
df[setdiff(names(df), "z")]         # drop unwanted
```

Filter rows with logical subsetting:

```
mtcars[mtcars$gear == 5, ]
mtcars[mtcars$gear == 5 & mtcars$cyl == 4, ]
```

Use `&` and `|` (not `&&` and `||`).

De Morgan's laws: `!(X & Y)` = `!X | !Y` and `!(X | Y)` = `!X & !Y`.

Boolean Algebra vs. Sets

Boolean (logical)	Set (integer via `which()`)
`x & y`	`intersect(x, y)`
`x | y`	`union(x, y)`
`x & !y`	`setdiff(x, y)`

`which()` converts logical to integer indices:

```
x <- c(FALSE, TRUE, FALSE, TRUE, TRUE)
which(x)
```

```
#> [1] 2 4 5
```

Key difference: logical subsetting preserves `NA`s; `which()` drops them.
Prefer integer subsetting when you need the first/last `TRUE` or have very few `TRUE`s in a long vector.

Exercises (Applications)

1. How would you randomly permute the columns of a data frame? Can you permute rows and columns simultaneously?
2. How would you select a random sample of `m` rows? What about a contiguous sample?
3. How could you put columns in alphabetical order?

Solutions (Applications)

1.

```
mtcars[sample(ncol(mtcars))]          # permute cols
mtcars[sample(nrow(mtcars)),
  sample(ncol(mtcars))]               # both
```

2. Random sample: `mtcars[sample(nrow(mtcars), m), ]`. Contiguous:

```
start <- sample(nrow(mtcars) - m + 1, 1)
mtcars[start:(start + m - 1), ]
```

3. `mtcars[order(names(mtcars))]` or `mtcars[sort(names(mtcars))]`.